TravelMate Trip Planner Website

Django Interview Preparation Guide

This document is a code-grounded explanation of the project in this repository. It is intended to help you explain the application clearly in an interview and to run a short live demo.

1. One-minute project explanation

TravelMate is a server-rendered Django MVT web application for discovering destinations, comparing hotels and transport options, viewing holiday packages, submitting trip booking requests, and reviewing private user history.

The application uses:

- Django 5.2 as the web framework.
- SQLite as the default relational database.
- Django templates for server-side HTML rendering.
- Django ORM for database access.
- Django built-in `User` authentication, sessions, CSRF protection, messages, and admin.
- Plain JavaScript for client-side interactions such as filters, autocomplete, mobile navigation, booking-form behavior, and browser-local saved trips.
- CSS and externally hosted image URLs for the frontend.

This is an MVT application, not a REST API application. There is no Django REST Framework dependency, serializer layer, API router, or JSON API endpoint in the current code.

2. Architecture at a glance

```
Browser
  |
  | HTTP request
  v
tripplanner/urls.py
  |
  +--> accounts/urls.py --> accounts/views.py
  |
  +--> travel/urls.py   --> travel/views.py
                              |
                              +--> forms.py
                              +--> models.py --> SQLite (db.sqlite3)
                              +--> templates/*.html
                              +--> Django messages/session/auth

Browser receives HTML
  |
  +--> static/travel/js/app.js for interactive behavior
  +--> static/travel/css/styles.css for presentation
```

The request cycle is:

1. The browser requests a URL.
2. Django's root URLconf chooses the included app URLconf.
3. A view reads query parameters or submitted form data.
4. The view validates data with a Django form when appropriate.
5. The view queries or changes the database through the ORM.
6. The view renders a template with a context dictionary, or redirects.
7. The browser renders the response and JavaScript adds client-side behavior.

3. Repository map

- manage.py: command-line entry point for Django tasks.
- tripplanner/settings.py: installed apps, middleware, templates, database, static files, authentication redirects, timezone, and security-related settings.
- tripplanner/urls.py: root URL routing. Includes admin, accounts, and travel routes.
- tripplanner/wsgi.py: WSGI entry point for traditional Python web servers.
- tripplanner/asgi.py: ASGI entry point for asynchronous-capable servers.
- travel/models.py: domain models and relationships.
- travel/forms.py: search form and booking ModelForm.
- travel/views.py: destination, search, detail, booking, and history-related application behavior.
- travel/urls.py: public travel URL patterns.
- travel/admin.py: Django admin registration and list/search/filter configuration.
- travel/migrations/: versioned database schema changes.
- travel/management/commands/seed_demo_data.py: custom command that creates demo inventory.
- accounts/views.py: signup, login, logout, and private history views.
- accounts/forms.py: custom signup and remember-me authentication forms.
- accounts/urls.py: account URL patterns.

- templates/: server-rendered HTML templates.
- static/travel/js/app.js: browser-side interactions.
- static/travel/css/styles.css: application styling.
- requirements.txt: currently only Django is required.
- README.md: short setup instructions.

4. URL and feature map

Root routes are mounted in tripplanner/urls.py:

URL	View	Purpose
`/`	`travel.views.home`	Homepage, global search, and featured inventory
`/search/<service>/`	`travel.views.service_search`	Search transport, hotels, or packages by service tab
`/hotels/`	`travel.views.hotels`	Hotel listing
`/hotels/<pk>/`	`travel.views.hotel_detail`	Hotel detail and browsing-history recording
`/transport/`	`travel.views.transport`	Transport listing
`/transport/<pk>/`	`travel.views.transport_detail`	Transport detail and browsing-history recording
`/packages/`	`travel.views.packages`	Holiday-package listing
`/packages/<pk>/`	`travel.views.package_detail`	Package detail and browsing-history recording
`/destinations/<pk>/`	`travel.views.destination_detail`	Destination detail plus related inventory
`/book/`	`travel.views.book_trip`	Authenticated booking request form
`/booking-success/`	`travel.views.booking_success`	Booking confirmation page
`/accounts/signup/`	`accounts.views.signup_view`	Create an account and log in
`/accounts/login/`	`accounts.views.login_view`	Authenticate a user
`/accounts/logout/`	`accounts.views.logout_view`	End the session
`/accounts/history/`	`accounts.views.history_view`	Authenticated booking and browsing history
`/admin/`	Django admin	Administer inventory and bookings

`<pk>` is a URL path converter for an integer primary key. Django passes it to the view as `pk`.

5. Domain model and database relationships

Destination

Stores a destination name, country, description, optional image URL, and best season. It is ordered alphabetically by name.

Hotel

Belongs to one `Destination` through a `ForeignKey`. A destination can have many hotels through the reverse name `destination.hotels`. It stores rating, price per night, room availability, amenities, and image URL.

TransportOption

Belongs to one destination. It stores source, destination, transport type, provider, departure and arrival time, price, and available seats. Transport type uses model choices: plane, train, bus, or cab.

HolidayPackage

Belongs to one destination. It stores title, duration, price per person, whether hotel and transport are included, itinerary, and image URL.

TripBooking

Stores a user's booking request. It has:

- A nullable `user` foreign key to Django's built-in `User` model.
- A required destination.
- Optional hotel, transport, or holiday package foreign keys.
- A `booking_type` choice: hotel, transport, or package.
- Customer contact information, date fields, traveler count, special request, and creation time.

The optional service relationships use `SET_NULL`, so deleting an inventory item does not delete the historical booking. The destination uses `PROTECT`, so a destination cannot be deleted while bookings refer to it. The user uses `CASCADE`, so deleting a user deletes that user's bookings.

BrowsingHistory

Stores a normalized snapshot of what an authenticated user opened: item type, item name, destination name, optional price and image URL, notes, and timestamp. It uses a foreign key to `User` and is ordered newest first.

Relationship summary

```
Destination 1 ---- many Hotel
Destination 1 ---- many TransportOption
Destination 1 ---- many HolidayPackage
Destination 1 ---- many TripBooking
User        1 ---- many TripBooking
User        1 ---- many BrowsingHistory
TripBooking many -- 0 or 1 Hotel
TripBooking many -- 0 or 1 TransportOption
TripBooking many -- 0 or 1 HolidayPackage
```

The booking model is polymorphic by convention: `booking_type` indicates which optional related service should be populated. A production design could enforce that combination with custom form/model validation or separate booking subtypes.

6. Main user flows

Flow A: Open the homepage

1. The browser requests `/`.
2. `home()` creates `TripSearchForm(request.GET or None)`.
3. The view starts querysets for destinations, hotels, transport, and packages.
4. If the GET form is valid, the view applies:
    - case-insensitive text matching with `icontains`;
    - `Q` objects for OR conditions;
    - transport-type filtering;
    - maximum-price filtering.
5. `select_related("destination")` reduces extra database queries for related destination data.
6. The view limits featured results with slices such as `destinations[:9]` and passes everything to `travel/home.html`.
7. The template extends `travel/base.html`, which loads static CSS and JavaScript and renders the navigation.

Flow B: Search by service

The homepage has service tabs such as flights, hotels, packages, trains, buses, and cabs. JavaScript changes the form action to `/search/<service>/` and enables the matching fields.

`service_search()` reads GET parameters such as `from`, `to`, `location`, `depart`, `return`, `travellers`, `travel_class`, `max_price`, `provider`, and `sort`.

The view then branches by service kind:

- Transport: filters `TransportOption` by transport type, source, destination, provider, and price.
- Hotel: filters `Hotel` by location/name and price.
- Package: filters `HolidayPackage` by destination/title and price.

It uses `Min()` to find the lowest price, orders by price/time/rating/duration, and sends results to `search_results.html`.

Important interview point: several search fields are currently presentation/demo inputs. For example, the dates, traveler count, and travel class are read and displayed but do not change the database query. The current inventory is demo data, not live airline or hotel-provider data.

Flow C: Open a detail page

1. A URL such as `/hotels/3/` reaches `hotel_detail(request, pk)`.
2. `get_object_or_404()` loads the object or returns HTTP 404 if the primary key does not exist.
3. `select_related("destination")` loads the related destination efficiently.
4. `record_browsing()` creates a `BrowsingHistory` row only when `request.user.is_authenticated` is true.
5. The view renders the detail template.

The same pattern is used for hotel, transport, package, and destination details. Destination details also use reverse relations such as `destination.hotels.all()`.

Flow D: Sign up and log in

Signup:

1. `/accounts/signup/` sends a GET request and displays `SignUpForm`.
2. A POST binds `request.POST` to the form.
3. `UserCreationForm` validates the username and matching passwords and hashes the password before saving.
4. The custom form also collects email and optional first name.
5. `login(request, user)` creates the authenticated session and redirects home.

Login:

1. `RememberMeAuthenticationForm` extends Django's `AuthenticationForm`.
2. A valid form identifies the user without manually checking password hashes.
3. `login()` stores the authenticated user in the session.
4. The remember-me checkbox sets a two-week expiry; otherwise `set_expiry(0)` makes the session expire when the browser closes.
5. `next` is used to return a user to the page that required login.

Flow E: Submit a booking

1. `/book/` is protected by `@login_required`.
2. A GET request pre-fills name and email from `request.user`.
3. A POST request binds `TripBookingForm(request.POST)`.
4. The `ModelForm` validates and maps form fields to the `TripBooking` model.
5. `form.save(commit=False)` creates the model instance without saving yet.
6. The view assigns `booking.user = request.user`, then calls `booking.save()`.
7. A success message is added with Django's messages framework.
8. The view redirects to `booking_success`, following the Post/Redirect/Get pattern and helping prevent duplicate resubmission on refresh.

The template includes `{% csrf_token %}`, which protects the POST against cross-site request forgery.

Flow F: View private history

1. `/accounts/history/` is protected by `@login_required`.
2. Bookings are filtered with `user=request.user`, so one user cannot see another user's rows through the normal view.
3. Related objects are loaded with `select_related()`.
4. Browsing history is also filtered by the current user and limited to the newest 30 rows.
5. `history.html` displays bookings and explored items.

Flow G: Save trips in the browser

`setupSavedTrips()` in static/travel/js/app.js uses `localStorage`. It stores saved names under a key containing the username, or `guest` for anonymous visitors. This is separate from Django's database-backed browsing history and bookings. It is browser/device-local and is not a server-side reservation.

7. Django concepts used

MVT

- Model: `travel/models.py` defines database-backed domain objects.
- View: Python functions receive `request`, perform application logic, and return a response.
- Template: HTML files render context using Django template tags and filters.

Django calls this pattern MVT, which is similar to MVC. Django's template system plays the view/presentation role, while the Python view coordinates the request.

URL routing

`path()` maps URL patterns to view callables. `include()` composes app-level URLconfs. Named URLs such as `name="book_trip"` are reversed in templates with `{% url 'book_trip' %}`, avoiding hard-coded links.

ORM and querysets

The project uses QuerySets instead of handwritten SQL:

- `Model.objects.all()` retrieves records.
- `filter()` narrows records.
- `Q()` combines OR conditions.
- `select_related()` performs SQL joins for foreign keys.
- `aggregate(Min(...))` computes a database aggregate.
- `order_by()` applies ordering.
- `get_object_or_404()` converts a missing record into a 404 response.
- QuerySets are lazy: the database query is generally evaluated when results are iterated, rendered, sliced, or otherwise consumed.

Forms and ModelForms

`TripSearchForm` is a regular form because it does not save a model. `TripBookingForm` is a `ModelForm`, so Django generates fields from `TripBooking` and validates model-compatible input. Form errors are rendered next to fields in the template.

Authentication and authorization

Django's built-in auth app provides users, password hashing, login/logout, sessions, and authentication forms. `@login_required` is authorization at the view boundary. Filtering history by `request.user` is object-level data isolation.

Middleware

Configured middleware includes security headers, sessions, common HTTP behavior, CSRF protection, authentication, messages, and clickjacking protection. Middleware runs around views and attaches capabilities such as `request.user` and `request.session`.

Templates and static files

Templates use inheritance with `{% extends %}` and blocks. `base.html` centralizes navigation, messages, CSS, JavaScript, and footer markup. `{% load static %}` resolves files under the configured static directories.

Admin

Models are registered with custom `ModelAdmin` classes. `list_display`, `list_filter`, and `search_fields` make inventory and booking management usable for staff through `/admin/`.

Migrations

`0001_initial.py` creates the original travel tables. `0002_user_history.py` adds the user relation to bookings and creates browsing history. Migrations are version-controlled schema changes and should be applied with `python manage.py migrate`.

Management command

`seed_demo_data` is a custom command discovered under `travel/management/commands/`. It creates destinations, hotels, transport options, and packages for a repeatable demo. It deletes existing bookings and destinations first, so it should be treated as a demo reset command rather than a production data import.

8. REST API concepts: what is and is not present

What is not present

This repository does not currently implement a REST API. Evidence:

- `requirements.txt` contains Django only, not `djangorestframework`.
- There is no `serializers.py`.
- There are no `APIView`, `ViewSet`, `@api_view`, or router registrations.
- The views return HTML through `render()` or redirects, not JSON responses.
- JavaScript does not use `fetch()` or AJAX to call application endpoints.

The application still has HTTP endpoints, but they are page endpoints in a server-rendered web application, not REST resource endpoints.

How to explain this in an interview

A strong answer is:

> The project uses Django's traditional server-rendered MVT architecture. It has URL endpoints for HTML pages and form submissions, but it does not expose a REST API. If an API were required, I would add Django REST Framework, serializers for the models, API views or viewsets, router URLs under `/api/`, authentication and permissions, pagination, filtering, and JSON responses.

REST design you could add later

Possible resources would be:

- `GET /api/destinations/`
- `GET /api/destinations/{id}/`
- `GET /api/hotels/?destination=goa&max_price=5000`
- `GET /api/transport-options/?destination=goa&transport_type=bus`
- `GET /api/packages/`
- `POST /api/bookings/`
- `GET /api/me/bookings/`

A DRF implementation would typically include:

1. A serializer that converts model instances to JSON and validates incoming JSON.
2. A `ModelViewSet` or generic API views.
3. URL routing with `DefaultRouter` or explicit `path()` entries.
4. authentication, such as session authentication for same-site use or token/JWT authentication for separate clients.
5. permission classes so users can only read their own bookings.
6. pagination, filtering, throttling, and consistent error responses.

REST concepts to know: resources, HTTP verbs, status codes, stateless requests, JSON representation, idempotency, authentication versus authorization, pagination, filtering, and versioning. Do not claim these are already implemented in this repository.

9. How to run the project step by step

From the repository root, which contains `manage.py`:

```
python -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
python manage.py check
python manage.py migrate
python manage.py seed_demo_data
python manage.py runserver
```

Open `http://127.0.0.1:8000/`.

For admin access:

```
python manage.py createsuperuser
```

Then open `http://127.0.0.1:8000/admin/`.

Useful diagnostic commands:

```
python manage.py showmigrations
python manage.py makemigrations --check
python manage.py shell
```

The current project uses SQLite at `db.sqlite3`. The seed command uses remote Unsplash image URLs, so those images require network access in the browser. The core pages and database data do not require a separate frontend build system.

10. Suggested live interview demo

1. Start the server and open the homepage.
2. Search for a destination or set a budget to show GET query parameters and ORM filtering.
3. Open a destination, hotel, or package detail page.
4. Explain `get_object_or_404()`, `select_related()`, reverse relations, and browsing history.
5. Create an account and explain Django password hashing and session login.
6. Open the booking page and show the authenticated-only route.
7. Change booking type to show JavaScript field behavior.
8. Submit a booking and explain ModelForm validation, `commit=False`, CSRF, messages, and redirect-after-POST.
9. Open My History and explain filtering by `request.user`.
10. Open the admin and show how staff can manage the same models.
11. Mention that saved trips use browser `localStorage`, while bookings and browsing history use SQLite.
12. Be explicit that this is not a REST API project in its current form.

11. Important design decisions and tradeoffs

- SQLite keeps local setup simple, but PostgreSQL would be better for production concurrency and deployment.
- Server-side rendering keeps the application simple and SEO-friendly, but a separate frontend or mobile client would benefit from a REST/JSON API.
- `select_related()` improves foreign-key reads, but list views should be profiled and use pagination as data grows.
- `DecimalField` is appropriate for prices because it avoids binary floating-point currency errors.
- Model `choices` keep allowed categories explicit and make display labels available through `get_*_display()`.
- `on_delete` behavior expresses data-retention rules: `PROTECT` for destinations, `SET_NULL` for optional inventory, and `CASCADE` for user-owned history.
- The booking process is a request submission, not a payment or inventory-locking transaction. It does not yet decrement rooms/seats or integrate with suppliers.

- The browsing history table stores snapshots rather than generic foreign keys. This is simple and preserves what the user saw, but it duplicates some display data.

12. Production improvements to discuss

These are not claims about the current implementation; they are good follow-up ideas if an interviewer asks what you would improve.

Security and deployment

- Move `SECRET_KEY` to an environment variable.
- Set `DEBUG = False` in production.
- Configure `ALLOWED_HOSTS`.
- Use HTTPS, secure cookies, HSTS, and a production WSGI/ASGI deployment.
- Validate or proxy remote images and add content security policy where appropriate.
- Keep dependencies pinned and run vulnerability checks.

Booking correctness

- Add model/form validation so booking type matches the selected hotel, transport, or package.
- Validate date order and prevent past travel dates.
- Check destination consistency across all selected objects.
- Use database transactions and row locking when reserving inventory.
- Add booking status, payment status, cancellation, and confirmation identifiers.
- Avoid double-booking by atomically checking and decrementing availability.

Scalability and maintainability

- Add indexes for common search fields and foreign keys where needed.
- Paginate hotel, transport, package, and history results.
- Add automated tests for models, forms, views, permissions, and booking edge cases.
- Use structured logging and error monitoring.
- Separate service/search logic from views if the domain grows.
- Add a real API with DRF if a mobile app or separate SPA needs the data.
- Use a production database and object storage/CDN for images.

UX and accessibility

- Add server-side validation messages for the conditional booking fields instead of relying only on JavaScript.
- Make filters shareable and persistent in the URL.
- Add loading/error/empty states for live integrations.
- Verify keyboard navigation, labels, focus behavior, and screen-reader output.

13. Likely interview questions and concise answers

Why Django for this project?

Django provides routing, ORM, forms, authentication, sessions, CSRF protection, templates, admin, and migrations in one coherent framework. That fits a data-driven booking prototype well.

What is the difference between `TripSearchForm` and `TripBookingForm`?

`TripSearchForm` is a plain form used to validate search input and does not save anything. `TripBookingForm` is a `ModelForm` generated from `TripBooking`, so it validates and can save a booking model.

Why use `commit=False`?

It creates the model instance without immediately inserting it into the database, allowing the view to attach `request.user` before saving.

Why use `select_related()`?

For foreign keys, it fetches related rows with a SQL join and avoids repeated queries while templates access fields such as `hotel.destination.name`.

What does `@login_required` do?

It blocks anonymous users from a view and redirects them to the configured login URL, usually preserving a `next` destination.

How is CSRF handled?

Django's CSRF middleware checks unsafe requests, and the booking template includes `{% csrf_token %}` in its POST form.

Is the booking a real reservation?

No. It is a validated trip request saved to the database. It does not yet process payment, contact external providers, lock inventory, or send confirmation email.

Does the application use REST?

Not currently. It uses HTML page endpoints and Django forms. REST concepts are relevant as a possible extension, but no REST framework/API implementation exists in this repository.

Where does the frontend logic live?

HTML structure is in Django templates, styles are in `static/travel/css/styles.css`, and browser interactions are in `static/travel/js/app.js`. The JavaScript does not replace the server; it enhances the rendered pages.

How does the app protect a user's history?

The history view requires authentication and filters both bookings and browsing records by `request.user`. The database relationships also associate each row with a specific user.

14. Honest limitations to mention

- There are no automated tests in the repository.
- Search is demo inventory search, not a live supplier integration.
- The booking form has optional related fields and does not enforce all cross-field business rules.
- Availability fields are displayed data; booking does not reserve or decrement stock.
- The app is configured for development with `DEBUG = True`, an insecure placeholder secret key, and empty `ALLOWED_HOSTS`.
- `localStorage` saved trips are device/browser-local, not synchronized account data.
- The current REST/API layer is absent.

15. Final 30-second closing statement

> TravelMate is a Django MVT trip-planning application. URLconfs route requests to function-based views, views validate input with Django forms, query related travel models through the ORM, and render templates. Users authenticate with Django sessions, can submit booking requests through a ModelForm, and can see only their own bookings and browsing history. The frontend uses static JavaScript for interaction and localStorage for saved trips. The project currently uses server-rendered HTML rather than Django REST Framework, so adding a mobile client would require serializers, JSON endpoints, permissions, and API routing.